Data Structures and Algorithms

INFORMATION
TECHNOLOGY

Objectives

- In this session, you will learn to:
  - Implement a doubly-linked list

Implementing a Doubly-Linked List

- Consider a sorted list of 1,00,000 numbers stored in a linked list.
- If you want to display these numbers in ascending order, what will you do?
  - Traverse the list starting from the first node.

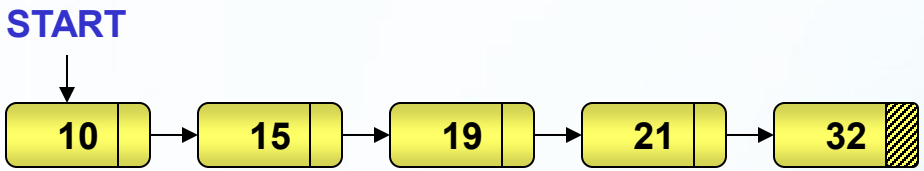
Implementing a Doubly-Linked List (Contd.)

- Now consider a case in which you need to display these numbers in a descending order.
- How will you solve this problem?
  - Each node is linked to the next node in sequence.
  - This means that you can traverse the list in the forward direction only.
  - Such a linked list is called a singly-linked list.
  - To display the numbers in the descending order, you need to reverse the linked list.

Data Structures and Algorithms

Implementing a Doubly-Linked List (Contd.)

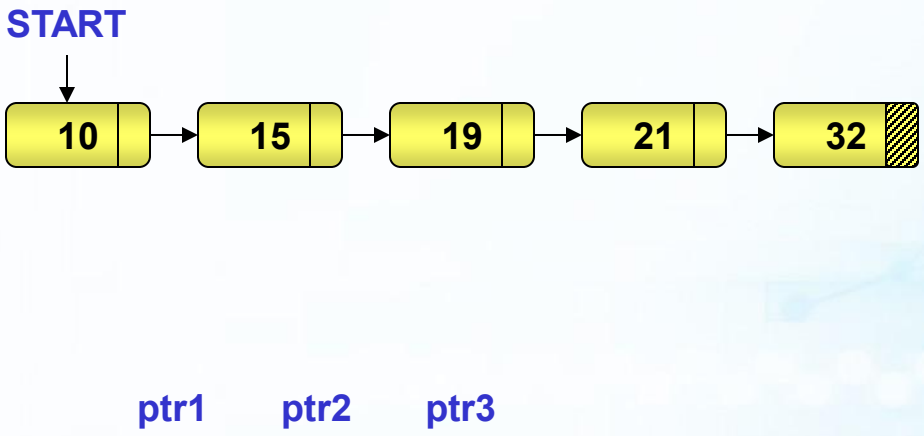
- Algorithm to reverse a singly-linked list:



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

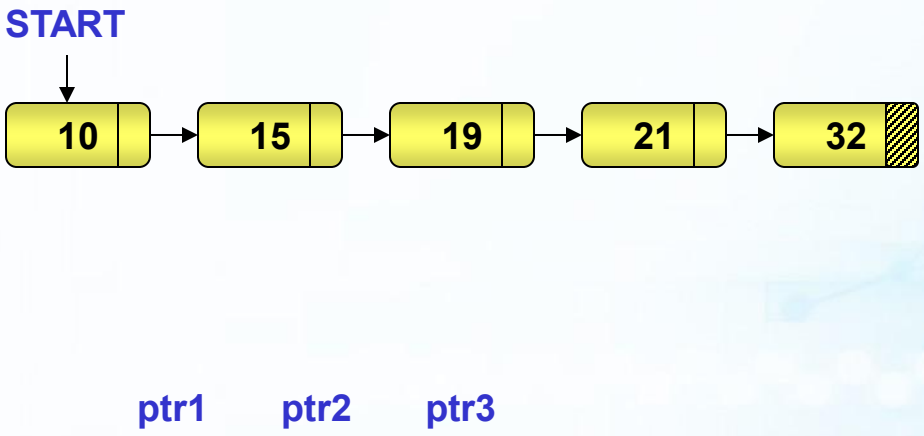
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

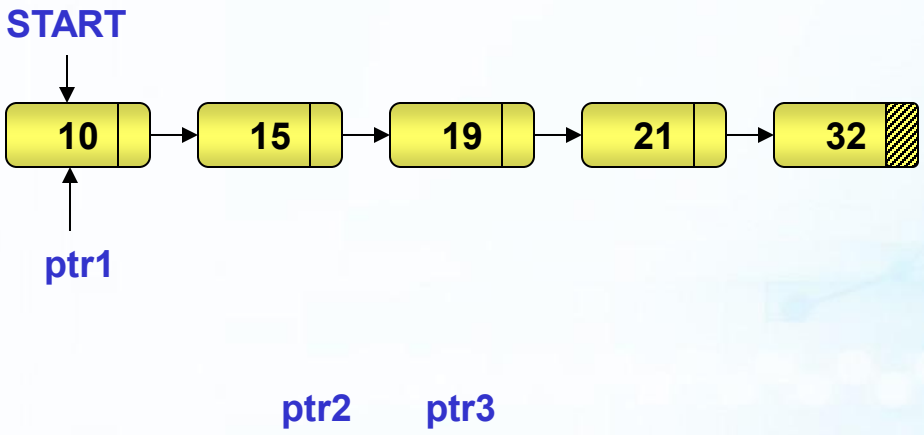
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

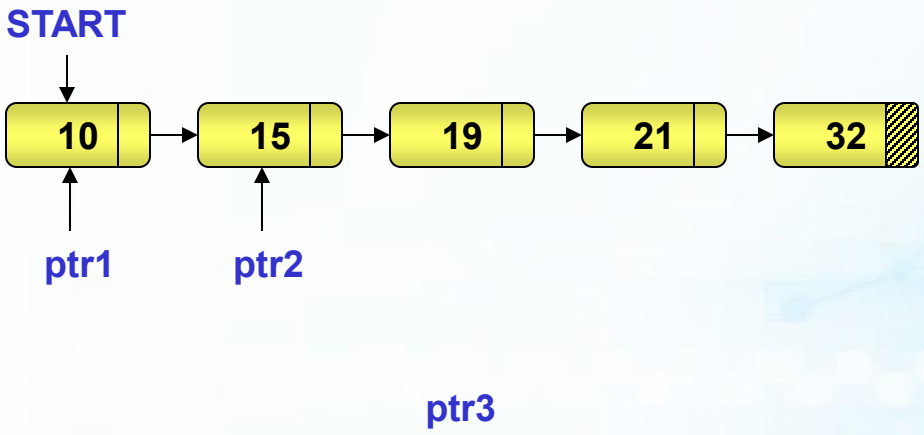
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

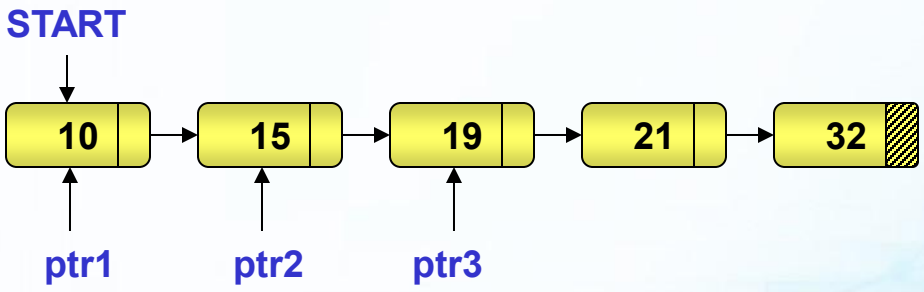
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

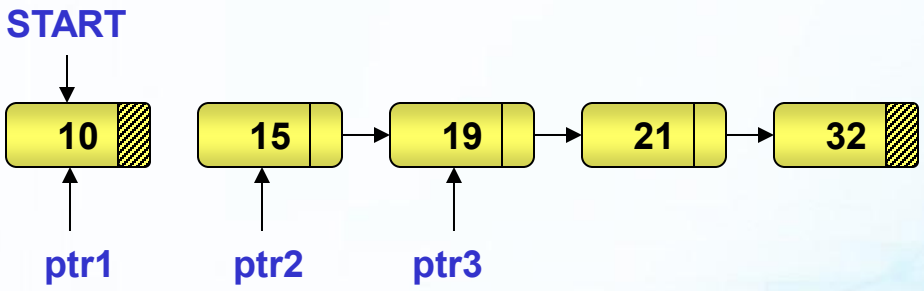
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

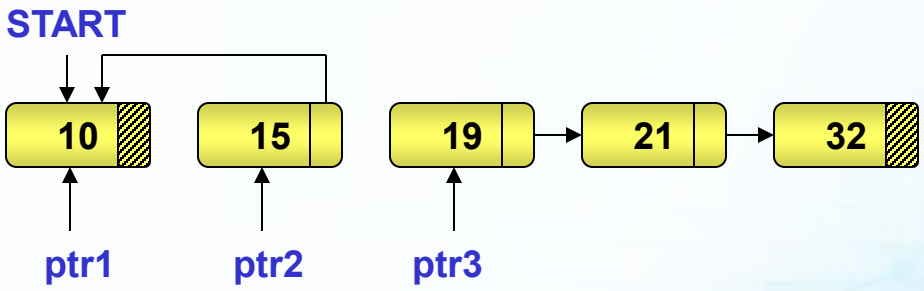
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

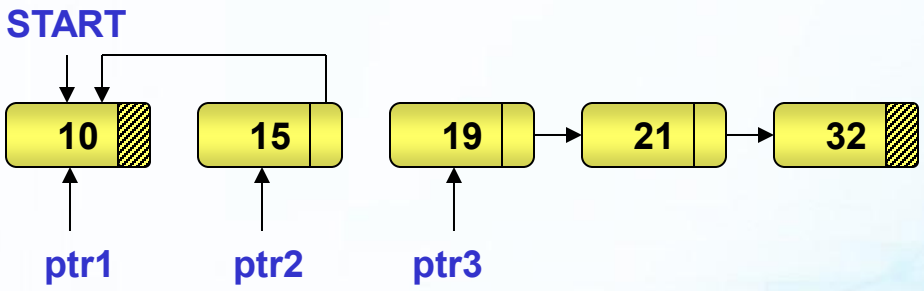
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

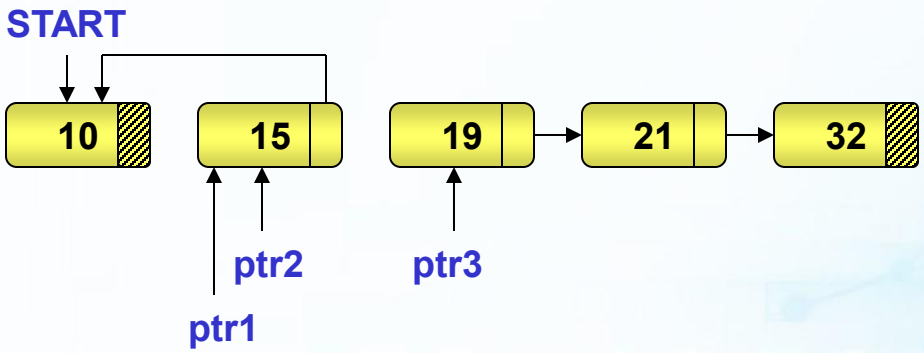
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

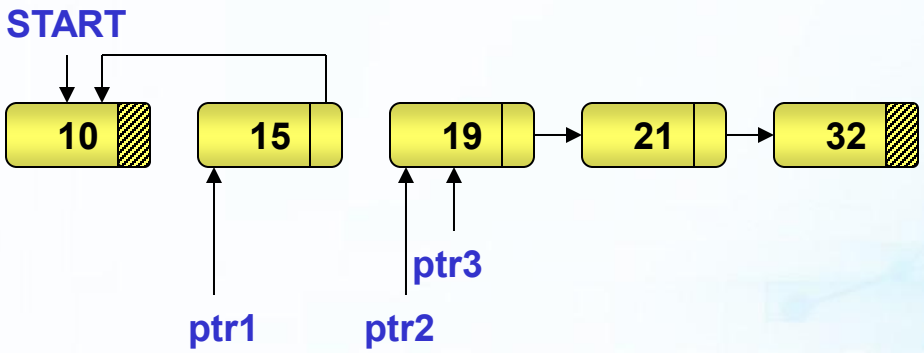
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

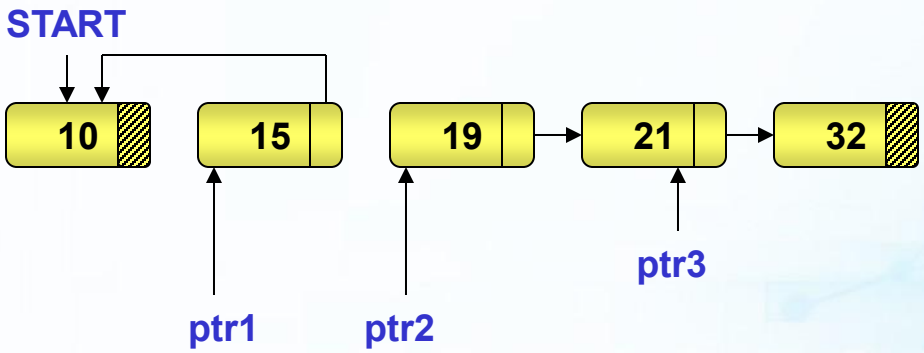
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

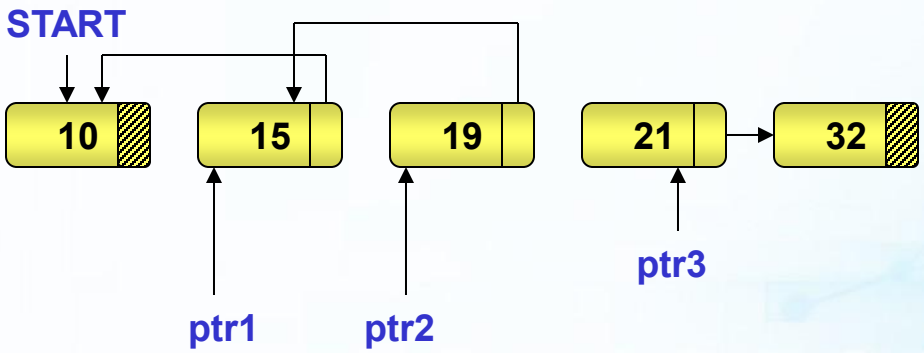
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

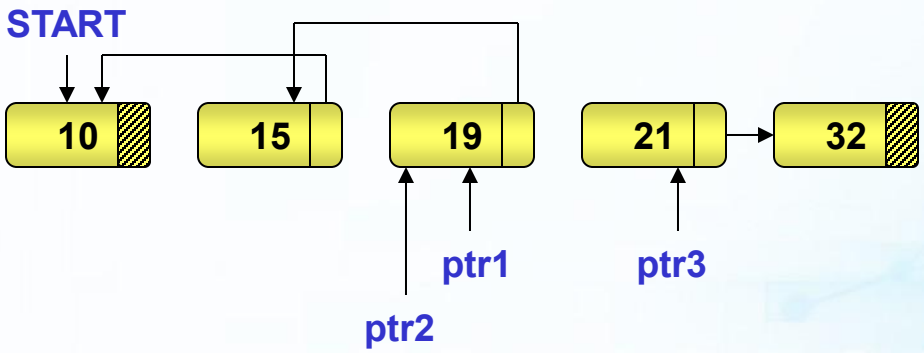
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

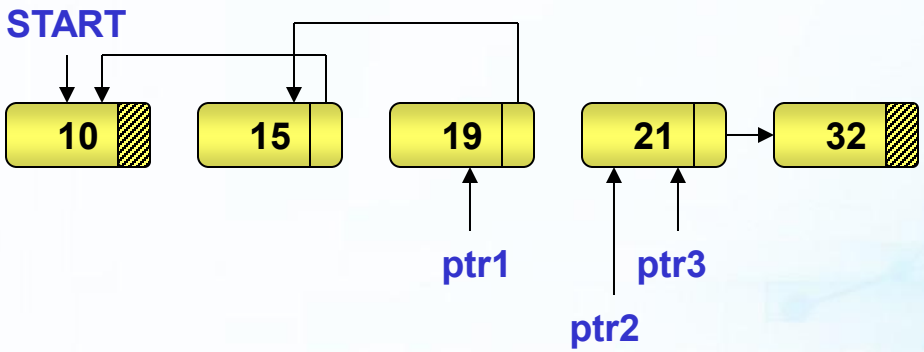
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

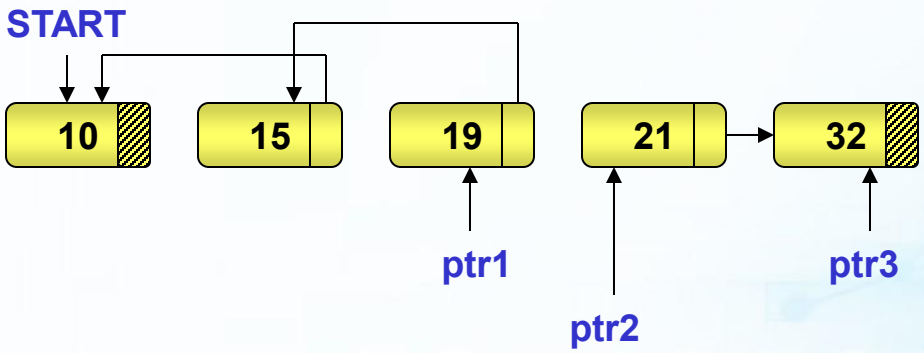
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

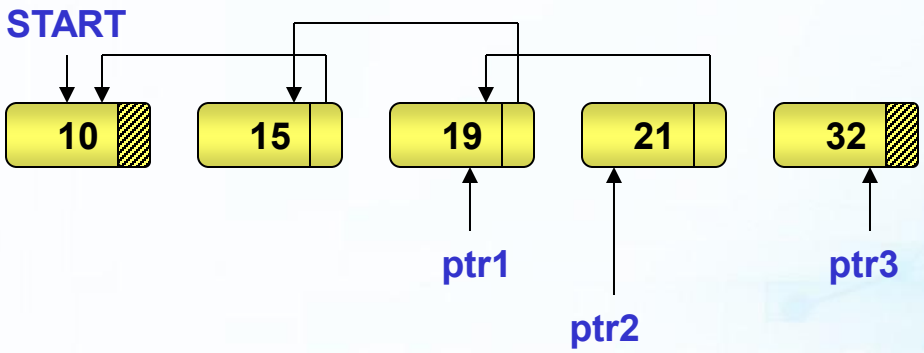
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

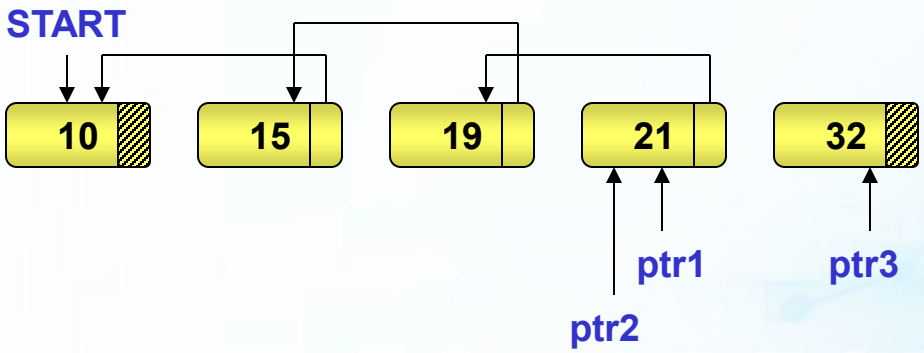
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

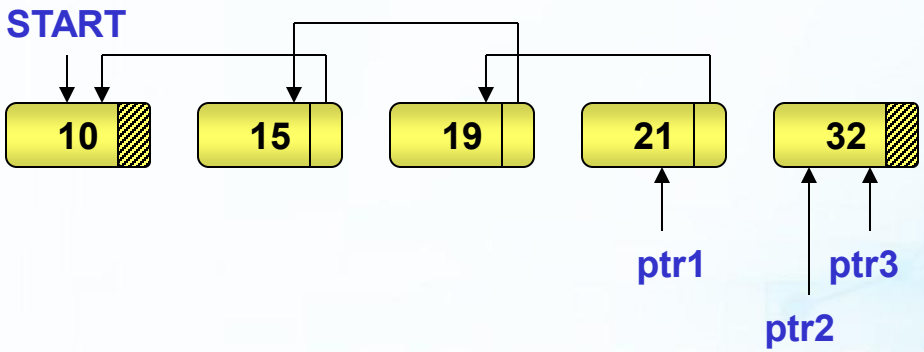
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

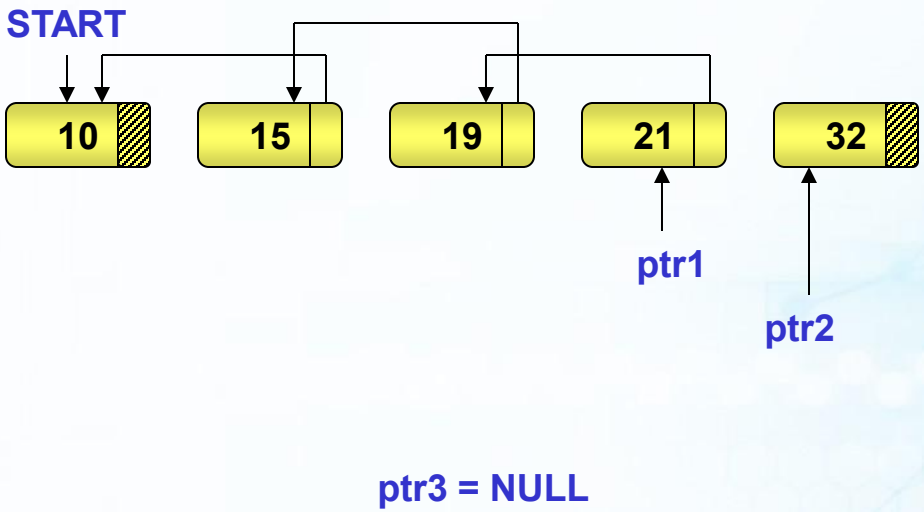
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

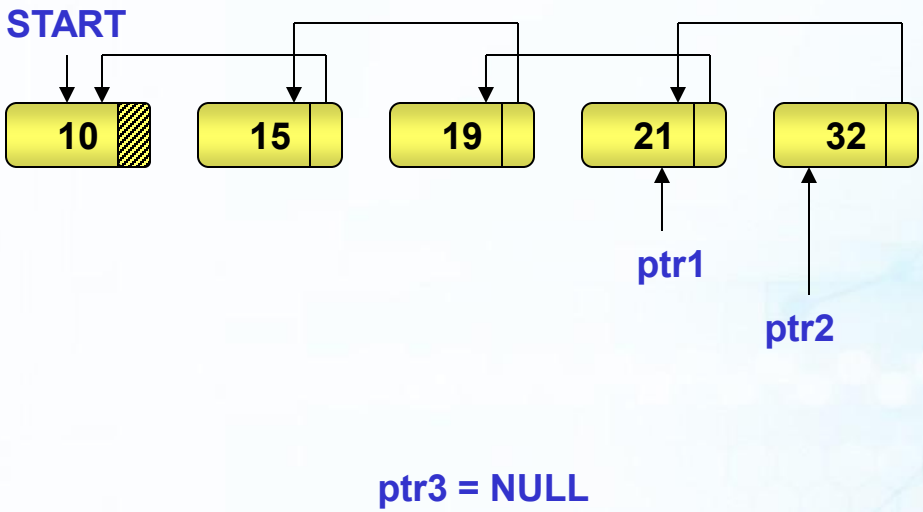
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

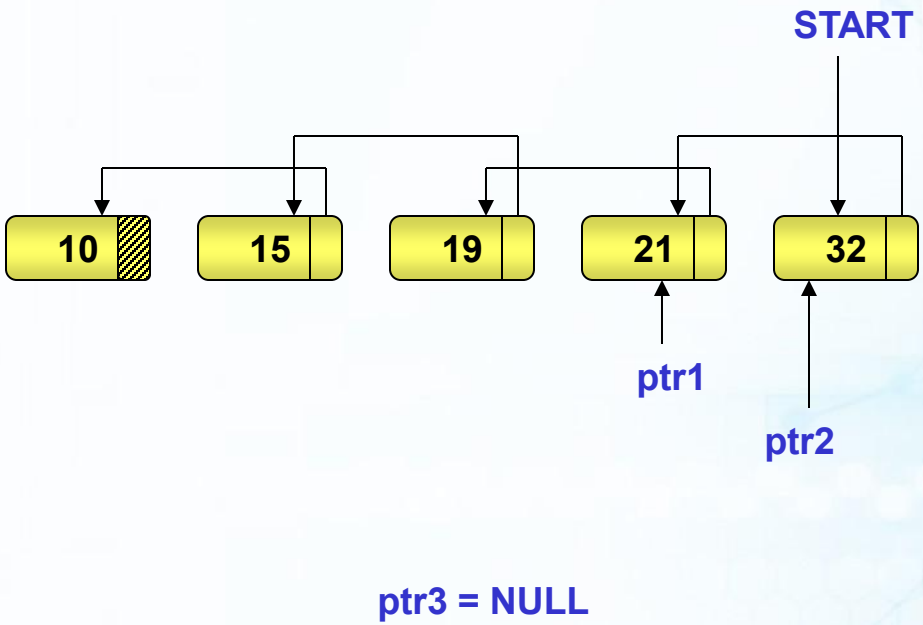
Implementing a Doubly-Linked List (Contd.)



1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Data Structures and Algorithms

Implementing a Doubly-Linked List (Contd.)



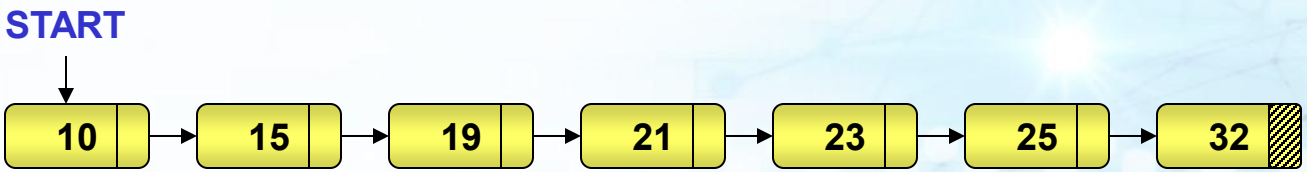
1. Declare three variables/pointers ptr1, ptr2, and ptr3.
2. If there is only one node in the list:
   Exit.
3. Mark the first node in the list as ptr1.
4. Mark the successor of ptr1 as ptr2.
5. Mark the successor of ptr2 as ptr3.
6. Make the next field of ptr1 point to NULL.
7. Make the next field of ptr2 point to ptr1.
8. Repeat until ptr3 becomes NULL:
   a. Set ptr1 = ptr2.
   b. Set ptr2 = ptr3.
   c. Make ptr3 point to the next node in its sequence.
   d. Make the next field of ptr2 point to ptr1.
9. Make START point to ptr2.

Implementing a Doubly-Linked List (Contd.)

- What is the problem with the previous algorithm?
  - You need to adjust the links for all the three variables whenever you visit the next node.
  - Disadvantage of this approach:
    - This approach is inefficient and time consuming for large lists.

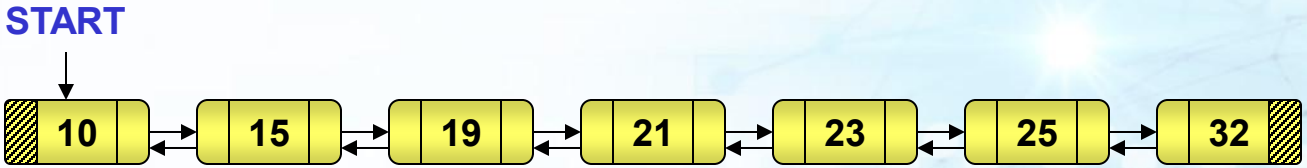
Implementing a Doubly-Linked List (Contd.)

- How can you solve this problem?
  - This problem can be solved if each node in the list holds the reference of its preceding node in addition to its next node in sequence.
  - Consider the following list:



Implementing a Doubly-Linked List (Contd.)

- You can introduce an additional field in each node of a singly-linked list, which would hold the address of its previous node.
- Such a type of list is known as a doubly-linked list.



Structure of a doubly-linked list

Representing a Doubly-Linked List

- A doubly-linked list is represented in a program by defining two classes:
  - `Node` class: In a doubly-linked list, each node needs to store:
    - The information
    - The address of the next node in sequence
    - The address of the previous node

```
// Code in Java:
class Node
{
   public int info;
   public Node next;
   public Node prev;
}
```

Data Structures and Algorithms

Representing a Doubly-Linked List (Contd.)

```
// Code in C++:
class Node
{
   public:
   int info;
   Node * next;
   Node * prev;
};
```

Representing a Doubly-Linked List (Contd.)

- `DoubleLinkedList` class:
  - Consists of a set of operations implemented on a linked list.
  - Declares a variable/pointer, `START`, which will always point to the first node in the list.

```
// Code in Java:
class DoubleLinkedList
{
 Node START;
 DoubleLinkedList(){}
 public void addNode(int element){}
 public bool search(int element){}
 public bool delNode(int element){}
 public void traverse() {}
 public void revtraverse(){}
}
```

Data Structures and Algorithms

Representing a Doubly-Linked List (Contd.)

```
// Code in C++:
class DoubleLinkedList
{
 Node *START;
 public:
 DoubleLinkedList();
 void addNode(int element);
 bool search(int element, Node *previous, Node
 *current);
 bool delNode(int element);
 void traverse();
 void revtraverse();
};
```

Just a Minute

- In doubly-linked list, which one of the following values is contained in the `prev` variable/pointer of the first node?
  - Address of the last node
  - Address of the second node
  - NULL

Just a Minute (Contd.)

- Solution:
  - NULL

Traversing a Doubly-Linked List

- Algorithm to traverse a doubly-linked list in the forward direction:

1. Mark the first node in the list as currentNode.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

Traversing a Doubly-Linked List (Contd.)

- Algorithm to traverse a doubly-linked list in the backward direction:

1. Mark the last node in the list as currentNode.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the node preceding it.

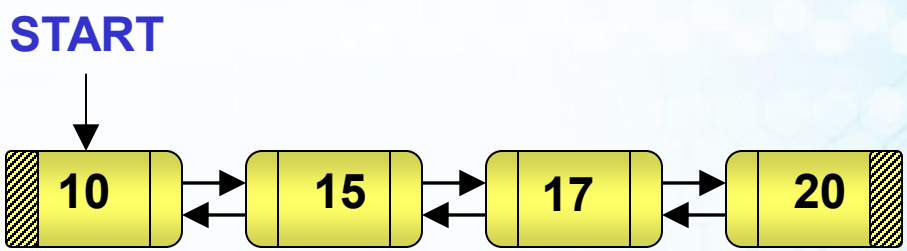
Inserting a Node in a Doubly-Linked List

- A node can be inserted at any of the following positions in a doubly-linked list:
  - Beginning of the list
  - Between two nodes in the list
  - End of the list

Data Structures and Algorithms

Inserting a Node in a Doubly-Linked List (Contd.)

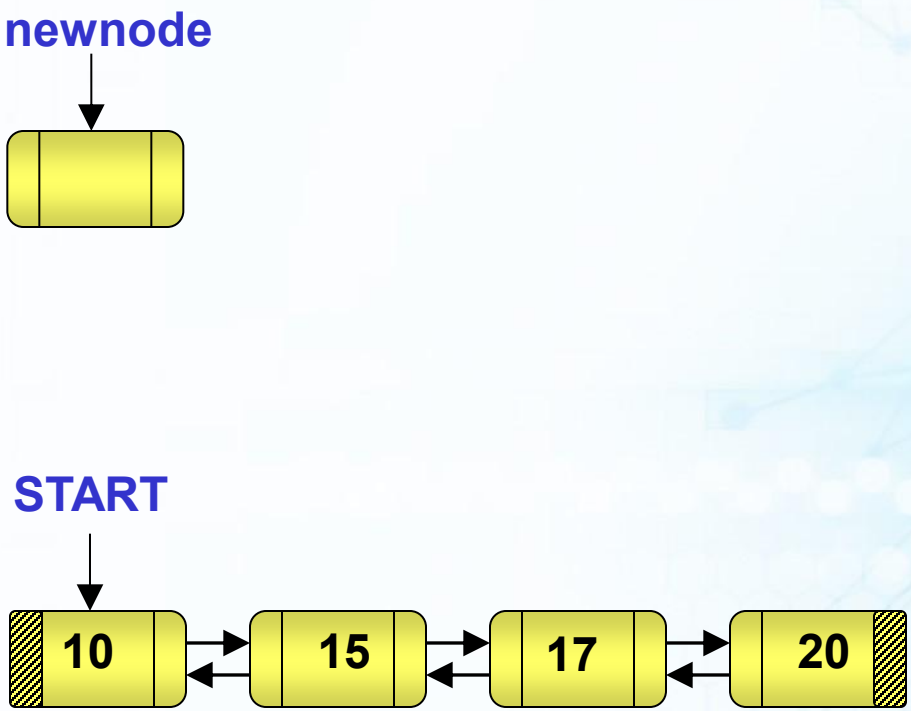
- Algorithm to insert a node at the beginning of a doubly-linked list:



1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. Make the next field of the new node point to the first node in the list.
4. Make the prev field of START point to the new node.
5. Make the prev field of the new node point to NULL.
6. Make START point to the new node.

Data Structures and Algorithms

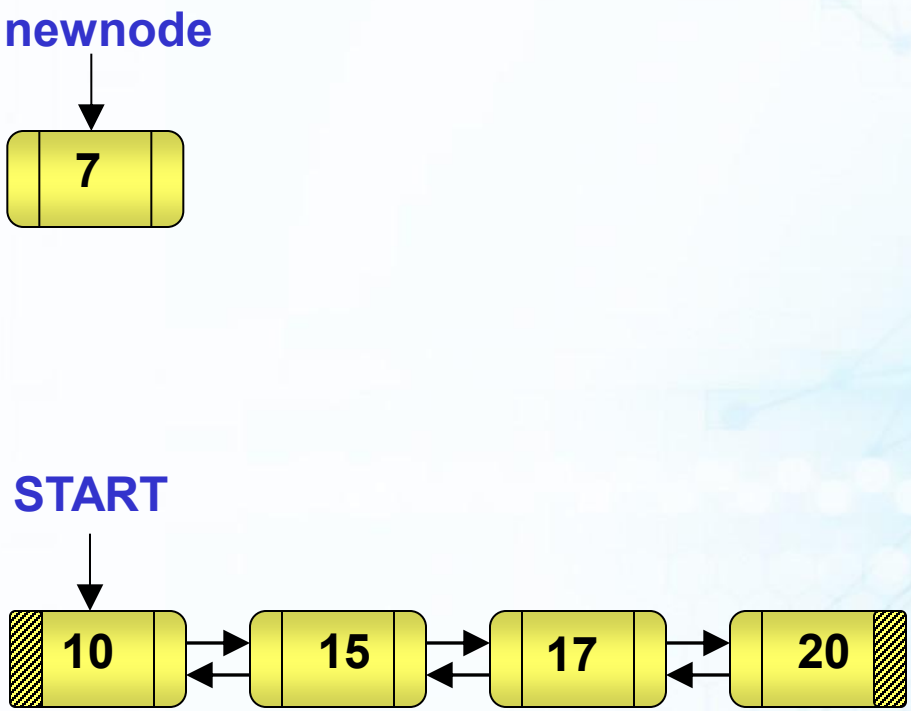
Inserting a Node in a Doubly-Linked List (Contd.)



1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. Make the next field of the new node point to the first node in the list.
4. Make the prev field of START point to the new node.
5. Make the prev field of the new node point to NULL.
6. Make START point to the new node.

Data Structures and Algorithms

Inserting a Node in a Doubly-Linked List (Contd.)

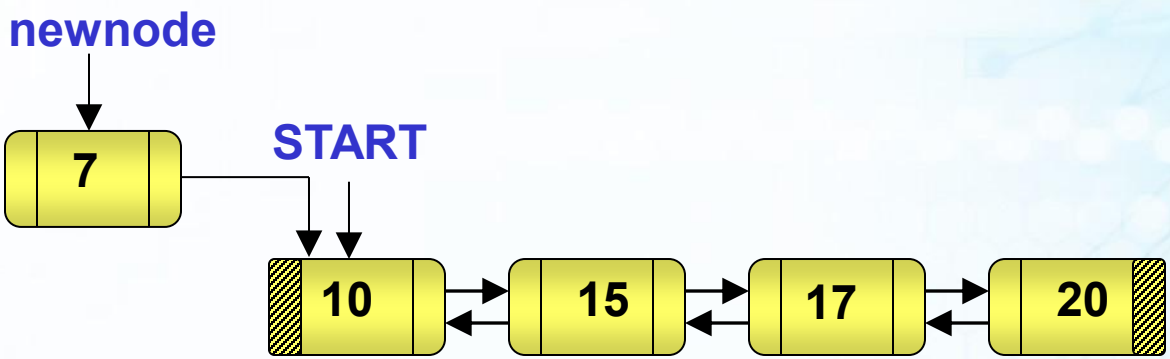


1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. Make the next field of the new node point to the first node in the list.
4. Make the prev field of START point to the new node.
5. Make the prev field of the new node point to NULL.
6. Make START point to the new node.

Data Structures and Algorithms

Inserting a Node in a Doubly-Linked List (Contd.)

newnode.next = START



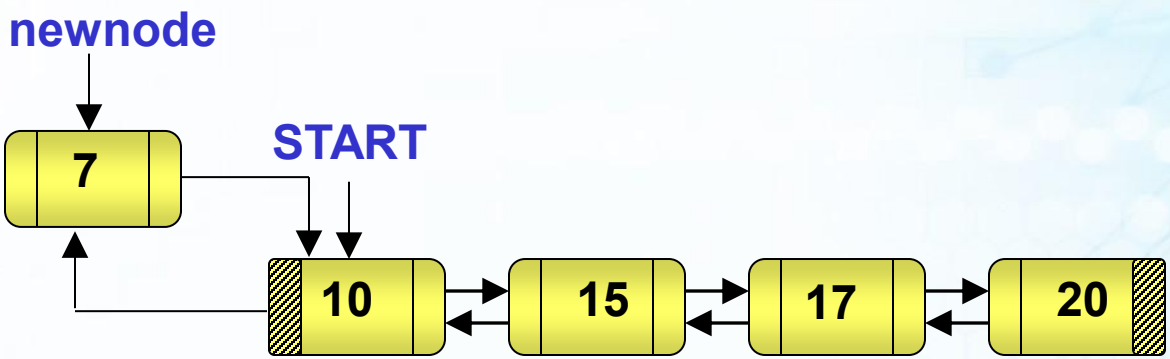
1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. Make the next field of the new node point to the first node in the list.
4. Make the prev field of START point to the new node.
5. Make the prev field of the new node point to NULL.
6. Make START point to the new node.

Data Structures and Algorithms

Inserting a Node in a Doubly-Linked List (Contd.)

newnode.next = START

START.prev = newnode



1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. Make the next field of the new node point to the first node in the list.
4. Make the prev field of START point to the new node.
5. Make the prev field of the new node point to NULL.
6. Make START point to the new node.

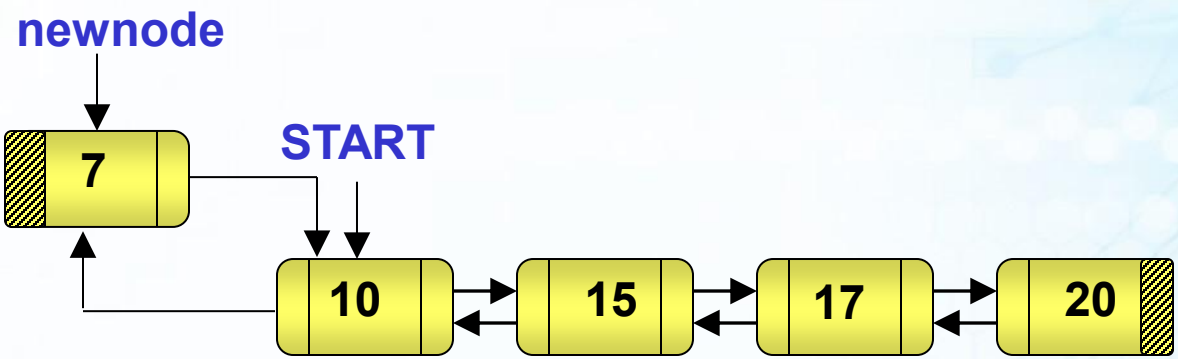
Data Structures and Algorithms

Inserting a Node in a Doubly-Linked List (Contd.)

newnode.next = START

START.prev = newnode

newnode.prev = NULL



1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. Make the next field of the new node point to the first node in the list.
4. Make the prev field of START point to the new node.
5. Make the prev field of the new node point to NULL.
6. Make START point to the new node.

Data Structures and Algorithms

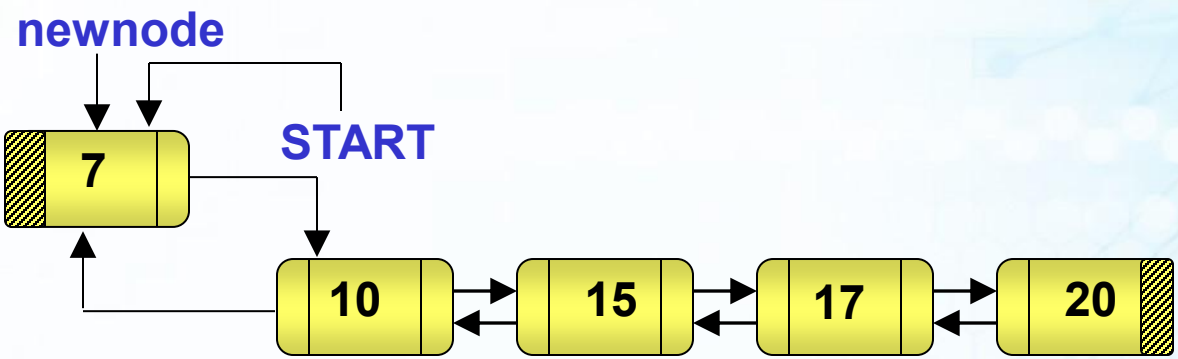
Inserting a Node in a Doubly-Linked List (Contd.)

newnode.next = START

START.prev = newnode

newnode.prev = NULL

START = newnode



Insertion complete

1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. Make the next field of the new node point to the first node in the list.
4. Make the prev field of START point to the new node.
5. Make the prev field of the new node point to NULL.
6. Make START point to the new node.

Inserting a Node in a Doubly-Linked List (Contd.)

- The following algorithm depicts the logic for inserting a node between two nodes in a doubly-linked list:

```
1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. If START is NULL (if the list is empty):
    a. Make the next field of the new node point to NULL.
    b. Make the prev field of the new node point to NULL.
    c. Make START point to the new node.
4. Identify the nodes between which the new node is to be
   inserted.
5. Mark them as previous and current, respectively. To
   locate previous and current, execute the following
   steps:
    a. Make current point to the first node.
    b. Make previous point to NULL.
```

Inserting a Node in a Doubly-Linked List (Contd.)

```
    c. Repeat the steps, d and e, until current.info >
       newnode.info or current = NULL.
    d. Make previous point to current.
    e. Make current point to the next node in the
       sequence.
6. Make the next field of the new node point to current
   and the prev field of the new node point to previous.
7. If current is not NULL:
     Make the prev field of current point to the new node.
8. If previous is not NULL:
     Make the next field of previous point to the new node.
9. If previous is NULL:
     Make START point to the new node.
```

Inserting a Node in a Doubly-Linked List (Contd.)

- Algorithm to insert a node at the end of a doubly-linked list that contains a variable, `LAST`, to keep track of the last node of the list:

```
1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. If LAST is not NULL:
      Make the next field of the node marked as LAST point
      to the new node.
4. Make the prev field of the new node point to the node
   marked LAST.
5. Make the next field of the new node point to NULL.
6. Mark the new node as LAST.
```

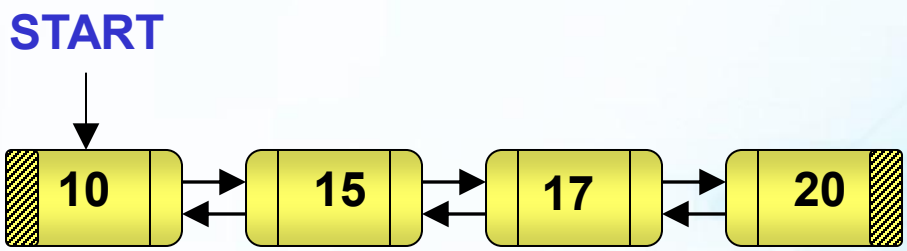
Deleting Nodes from a Doubly-Linked List

- You can delete a node from one of the following places in a doubly-linked list:
  - Beginning of the list
  - Between two nodes in the list
  - End of the list

Data Structures and Algorithms

Deleting Nodes from a Doubly-Linked List (Contd.)

- Algorithm to delete a node from the beginning of the doubly-linked list:

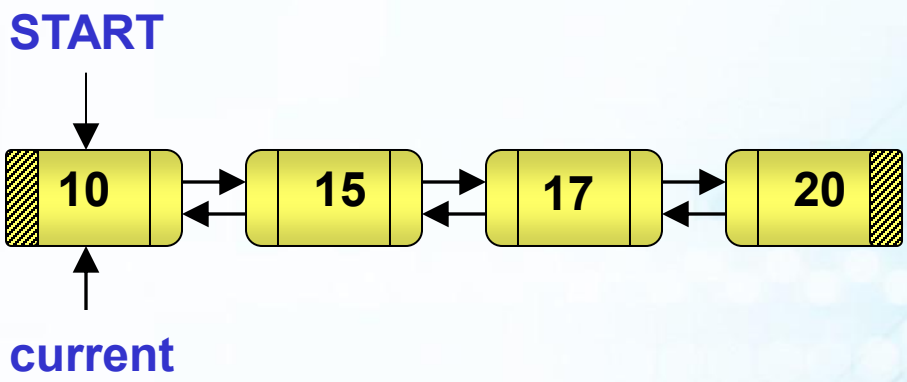


1. Mark the first node in the list as current.
2. Make START point to the next node in the sequence.
3. If START is not NULL:
   /* If the node to be deleted is not the only node in the list */

   Assign NULL to the prev field of the node marked as START.
4. Release the memory of the node marked as current.

Data Structures and Algorithms

Deleting Nodes from a Doubly-Linked List (Contd.)



1. Mark the first node in the list as current.

2. Make START point to the next node in the sequence.

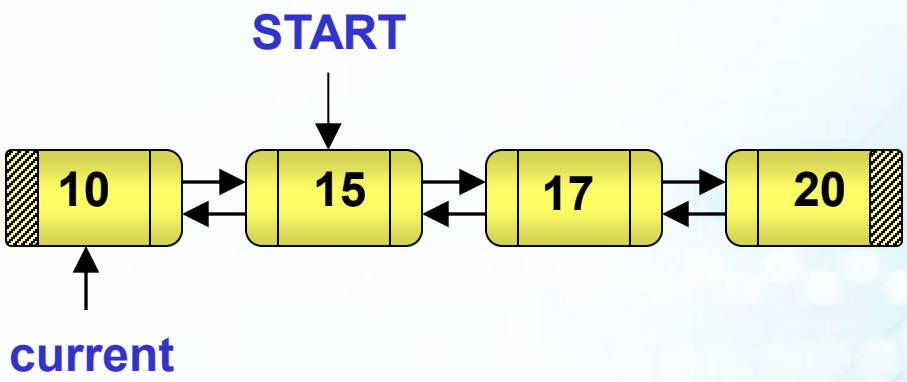
3. If START is not NULL:
   /* If the node to be deleted is not the only node in the list */

   Assign NULL to the prev field of the node marked as START.

4. Release the memory of the node marked as current.

Data Structures and Algorithms

Deleting Nodes from a Doubly-Linked List (Contd.)



START = START.next

1. Mark the first node in the list as current.

2. Make START point to the next node in the sequence.

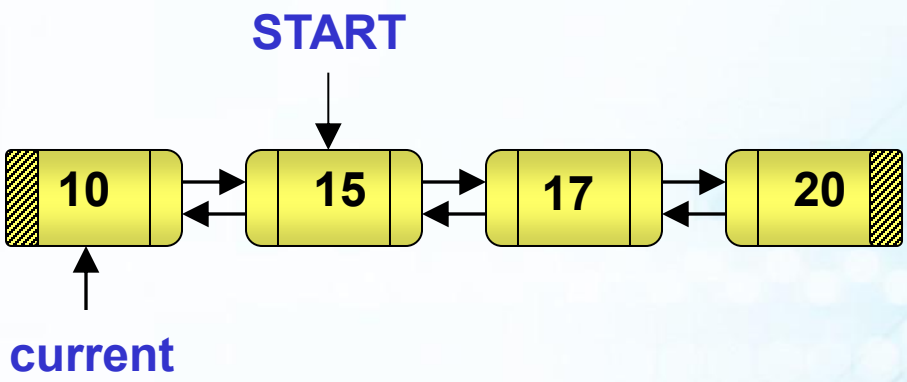
3. If START is not NULL:
   /* If the node to be deleted is not the only node in the list */

   Assign NULL to the prev field of the node marked as START.

4. Release the memory of the node marked as current.

Data Structures and Algorithms

Deleting Nodes from a Doubly-Linked List (Contd.)



1. Mark the first node in the list as current.

2. Make START point to the next node in the sequence.

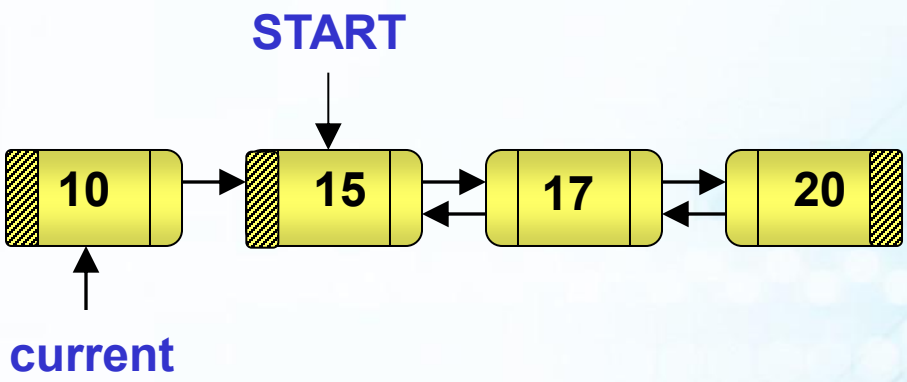
3. If START is not NULL:
   /* If the node to be deleted is not the only node in the list */

   Assign NULL to the prev field of the node marked as START.

4. Release the memory of the node marked as current.

Data Structures and Algorithms

Deleting Nodes from a Doubly-Linked List (Contd.)



START. prev = NULL

1. Mark the first node in the list as current.

2. Make START point to the next node in the sequence.

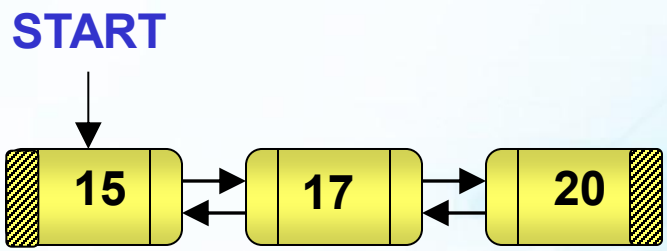
3. If START is not NULL:
   /* If the node to be deleted is not the only node in the list */

   Assign NULL to the prev field of the node marked as START.

4. Release the memory of the node marked as current.

Data Structures and Algorithms

Deleting Nodes from a Doubly-Linked List (Contd.)



Memory released

1. Mark the first node in the list as current.

2. Make START point to the next node in the sequence.

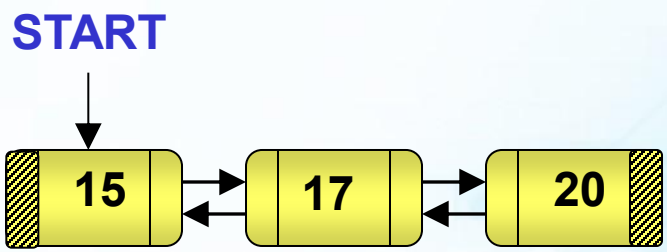
3. If START is not NULL:
   /* If the node to be deleted is not the only node in the list */

   Assign NULL to the prev field of the node marked as START.

4. Release the memory of the node marked as current.

Deleting Nodes from a Doubly-Linked List (Contd.)

Delete operation complete



1. Mark the first node in the list as current.
2. Make START point to the next node in the sequence.
3. If START is not NULL:
   /* If the node to be deleted is not the only node in the list */

   Assign NULL to the prev field of the node marked as START.
4. Release the memory of the node marked as current.

Deleting Nodes from a Doubly-Linked List (Contd.)

- The following algorithm deletes a node between two nodes and from the end of the doubly-linked list:

```
1. Mark the node to be deleted as current and its
   predecessor as previous. To locate previous and
   current, execute the following steps:
     a. Make previous point to NULL.
     b. Make current point to the first node in the
        linked list(that is, Set current = START).
     c. Repeat the steps, d and e, until either the value
        of current is same as the value to be deleted or
        current becomes NULL.
     d. Make previous point to current.
     e. Make current point to the next node in the
        sequence.
2. If current is NULL:
     a. Display "Value to be deleted not found in the
        list".
     b. Exit.
```

Deleting Nodes from a Doubly-Linked List (Contd.)

```
3. If previous is not NULL:
    Make the next field of previous point to the
    successor of current.
4. If previous is NULL:
    Make START point to its successor.
5. If the successor of current exists:
    Make the prev field of the successor of current
    point to previous.
6. Release the memory of the node marked as current.
```

Just a Minute

- To which one of the following options does the next pointer of the last node in a doubly-linked list point?
  - NULL
  - First node
  - START

Just a Minute (Contd.)

- Solution:
  - NULL

Summary

- In this session, you learned that:
  - A doubly-linked list in a program can be represented by declaring the following classes:
    - A class that represents a node in a doubly-linked list
    - A class that represents a doubly-linked list